Prompt 2 — Player App

Prompt 2 — Player App (Kotlin/Android)Đ
Smart School Bell — Live Announcement WebRTC UpgradeĐ
Đ
Use this prompt in Cursor inside your Player / Device App repository.Đ
Đ
---Đ
Đ
TaskĐ
Đ
Upgrade Live Announcement to WebRTC (Player / Device App).Đ
Đ
---Đ
Đ
ContextĐ
Đ
This is the **Player App** (Android device/speaker) for Smart School Bell System. Backend upgraded from WebSocket PCM relay to **WebRTC P2P** live announcement.Đ
Đ
- **Backend:** `https://api.shikkhasomoy.com`Đ
- **Auth:** JWT from login (`Authorization: Bearer <token>`)Đ
- **Swagger:** `https://api.shikkhasomoy.com/api-docs` !' Live tagĐ
Đ
IMPORTANT: Live audio is received via WebRTC directly from Controller. Server only does signaling — no audio through REST or WebSocket.Đ
Đ
---Đ
Đ
What to REMOVE (old system)Đ
Đ
Find and remove:Đ
Đ
- WebSocket binary PCM receive + play logic for live micĐ
- `AudioTrack` / buffer playback from WebSocket chunks for live announcementĐ
- Old handler for `live_start` that expects binary audio stream from WSĐ
- Any code that plays live audio from WebSocket binary messagesĐ
Đ
Do NOT remove or break:Đ
Đ
- Bell schedule sync (`GET /api/schedules`)Đ
- Azan sync (`GET /api/azan`)Đ
- Sounds download (`GET /api/sounds/bell`, `/azan`)Đ
- Recorded/scheduled announcements (`GET /api/announcements/*`)Đ
- Device heartbeat (`POST /api/device/heartbeat`)Đ
- WebSocket handler for `new_announcement` (scheduled push) — **KEEP**Đ
- Login, foreground service, existing alarm logicĐ
Đ
---Đ
Đ

What to ADDÐ
Ð
1. Dependencies (build.gradle)Ð
Ð
```gradleÐ
implementation 'io.getstream:stream-webrtc-android:1.1.3'Ð
// OR: org.webrtc:google-webrtcÐ
```Ð
Ð
**Permissions:**Ð
Ð
- `INTERNET`Ð
- `MODIFY_AUDIO_SETTINGS`Ð
- `WAKE_LOCK` (if already used)Ð
Ð
---Ð
Ð
### 2. Retrofit API — LiveApiService.ktÐ
Ð
| Method | Path | Body |Ð
|--------|------|------|Ð
| GET | `/api/live/config` | — |Ð
| GET | `/api/live/session?role=player&session_id={id}` | — |Ð
| POST | `/api/live/answer` | `{ "session_id", "role": "player", "answer": { "sdp", "type": "answer" }, "device_id" }` |Ð
| POST | `/api/live/ice` | `{ "session_id", "role": "player", "candidate", "sdpMid", "sdpMLineIndex" }` |Ð
| GET | `/api/live/ice?session_id={id}&role=player` | — |Ð
| POST | `/api/live/end` | `{ "session_id" }` |Ð
Ð
**Player does NOT call POST `/api/live/session`** (controller only).Ð
Ð
---Ð
Ð
### 3. WebSocket — notifications only (NO binary audio)Ð
Ð
```Ð
wss://api.shikkhasomoy.com/ws/announce?token={JWT}&role=userÐ
```Ð
Ð
(`role=player` also works)Ð
Ð
**JSON messages:**Ð
Ð
| type | Action |Ð
|------|--------|Ð
| `new_announcement` | **KEEP** — existing scheduled announcement logic |Ð
| `live_session` | `{ session_id, ice_servers }` ’ prepare WebRTC receiver |Ð
| `live_offer` | `{ session_id, offer: { sdp, type } }` ’ create answer, start P2P |Ð
| `live_ice` | `{ session_id }` ’ poll GET `/api/live/ice?role=player` |Ð
| `live_end` | `{ session_id, reason }` ’ teardown, hide live indicator |Ð
Ð
**Do NOT expect binary audio on WebSocket.**Ð

Đ
---Đ
Đ
### 4. WebRTC Receiver — LiveWebRtcPlayer.ktĐ
Đ
**Auto-receive flow (when `live_offer` received):**Đ
Đ
1. Get `ice_servers` from `live_session` or GET `/api/live/config`Đ
2. Create `PeerConnection` with ICE serversĐ
3. `setRemoteDescription(offer)` from `live_offer`Đ
4. `createAnswer()` !' `POST /api/live/answer`Đ
5. On local ICE candidate !' `POST /api/live/ice`Đ
6. On `live_ice` WS or poll !' GET `/api/live/ice?role=player` !' `addIceCandidate()`Đ
7. On remote audio track !' route to **speaker** (speakerphone ON)Đ
8. Auto-play — no user tap neededĐ
Đ
**Audio output:**Đ
Đ
- Use WebRTC remote audio track (not manual PCM buffer)Đ
- `AudioManager.requestAudioFocus()` before playĐ
- Speaker volume max for school PA useĐ
Đ
**On `live_end`:**Đ
Đ
- Close PeerConnectionĐ
- Release audio focusĐ
- Hide live indicatorĐ
Đ
**Auto reconnect:**Đ
Đ
- If DISCONNECTED/FAILED !' poll GET `/api/live/session?role=player` for 30sĐ
- If new session !' rejoin; else idleĐ
Đ
---Đ
Đ
### 5. Background ServiceĐ
Đ
If app uses `AnnouncementService` / foreground service:Đ
Đ
- Integrate `LiveWebRtcPlayer` into existing serviceĐ
- Keep WebSocket alive in serviceĐ
- WebRTC runs inside foreground service (Android 9+)Đ
- Do not break bell/azan alarm schedulingĐ
Đ
---Đ
Đ
### 6. UI (minimal)Đ
Đ
- Overlay or notification: "Live Announcement" when connectedĐ
- Status: Receiving / Connected / ReconnectingĐ
- No user interaction needed (auto-play)Đ

Đ
---Đ
Đ
### 7. Flow DiagramĐ
Đ
```Đ
[Background] WebSocket connected role=userĐ
  !“Đ
WS: live_session { session_id, ice_servers }Đ
  !“Đ
WS: live_offer { session_id, offer }Đ
  !“Đ
WebRTC: setRemoteDescription(offer) !’ createAnswer()Đ
  !“Đ
POST /api/live/answerĐ
  !“Đ
ICE exchange (POST + GET /api/live/ice)Đ
  !“Đ
Remote audio track !’ Speaker (auto play)Đ
  !“Đ
WS: live_end !’ teardownĐ
```Đ
Đ
---Đ
Đ
## RulesĐ
Đ
- Match existing project patterns (Retrofit, service, WS client)Đ
- Reuse existing JWT + device_id from login/heartbeatĐ
- Keep `new_announcement` WebSocket handler unchangedĐ
- Bell, azan, recorded announcement must still workĐ
- Test on real Android device with speakerĐ
Đ
---Đ
Đ
## DeliverablesĐ
Đ
1. `LiveApiService.kt`Đ
2. `LiveWebRtcPlayer.kt` (WebRTC receiver)Đ
3. Updated WebSocket message handler (JSON only for live)Đ
4. Updated AnnouncementService (if exists)Đ
5. Remove all old PCM WebSocket playback codeĐ
Đ
---Đ
Đ
## First Step for CursorĐ
Đ
Search codebase for existing live/WebSocket/PCM/audio playback code and list files to change. Then implement step by step without breaking bell/azan/announcements.Đ
Đ
---Đ

Đ
*School Bell Backend v2.0 — WebRTC Signaling API*Đ